Comparative Analysis of TCP on Windows and Linux

Alessandro Claudio Damo
University of Padua
Email: alessandroclaudio.damo@studenti.unipd.it

I. Introduction

Modern operating systems provide a certain degree of flexibility in the selection of the TCP congestion control algorithm; however, this flexibility differs significantly across platforms.

In the Linux environment, the TCP stack offers a wide range of congestion control algorithms that can be selected and configured at the system level. In addition to traditional loss-based approaches such as CUBIC and Reno, Linux also supports more recent and experimental solutions, including model-based algorithms such as BBR.

In contrast, the Windows environment provides a more limited set of options. The Windows TCP stack exposes only a small number of congestion control algorithms, with CUBIC being the default and officially supported choice.

To make the comparison more meaningful and differentiated, and to avoid a redundant evaluation of conceptually similar solutions, the simulation that we had on ns3 had BBR as the TCP congestion control algorithm in the Linux environment and CUBIC in the Windows environment.

The choice of CUBIC for Windows is largely mandatory, as it represents the congestion control algorithm implemented and officially adopted by the Windows TCP stack. As a result, it constitutes a realistic and representative configuration of typical Windows deployments.

For Linux, on the other hand, the TCP stack offers substantial flexibility in the selection of the congestion control algorithm, providing multiple alternatives. In order to significantly differentiate the comparison and to complement a loss-based approach with a fundamentally different design paradigm, BBR was selected. BBR is a more recent and experimental algorithm based on an explicit model of the bottleneck bandwidth and the minimum round-trip time. This choice enhances the value of the analysis by enabling a comparison not only between two different operating systems, but also between two fundamentally different congestion control philosophies. Obviously, since TCP CUBIC can also be selected on Linux, the Linux results are fully comparable to those obtained on Windows for all experiments simulated in ns-3.

An additional comparison is performed with respect to loss recovery policies, this time with real experiment where real TCP implementations of Windows and Linux are evaluated under controlled packet loss conditions. This analysis aims to observe not only performance trends, but also to infer the recovery strategies employed by the two TCP stacks.

II. Theoretical Background

1) Congestion Window and Receive Window on TCP: The transmission rate control in TCP is governed by the interaction between two fundamental windows: the **congestion window (cwnd)** and the **receive window (rwnd)**.

The receive window (*rwnd*) is a value advertised by the receiver and represents the maximum amount of data that it can buffer. This value is generally fixed by the receiver operating system and reflects application-level or memory constraints, rather than network congestion conditions.

The congestion window (*cwnd*), on the other hand, is a dynamic parameter maintained by the sender. It limits the amount of data that the sender can have in flight, i.e., transmitted but not yet acknowledged. The value of *cwnd* is dynamically adjusted by the congestion control algorithm based on the observed network conditions.

In general, TCP follows a continuous decision-making cycle:

1) the sender observes the current values of *cwnd* and *rwnd*;
2) it determines the maximum amount of data in flight as
$$\text{flight_size} \leq \min(\text{cwnd}, \text{rwnd});$$
3) new data are transmitted only if this condition is satisfied; otherwise, the sender waits for incoming ACKs.

The achievable throughput of a TCP flow is approximately proportional to the ratio between *cwnd* and the path RTT:
$$\text{throughput} \approx \frac{\text{cwnd}}{\text{RTT}}.$$
As a consequence, a larger congestion window enables higher throughput, while a reduction of *cwnd*, which is typical during recovery phases, leads to a decrease in the transmission rate.

A. TCP CUBIC

TCP CUBIC is a **loss-based** congestion control algorithm designed to improve TCP scalability over high-capacity links and in networks characterized by large round-trip times (RTTs).

Its main characteristic is that the growth of the **congestion window (cwnd)** does not depend on the number of received ACKs, as in traditional mechanisms, but **solely on the time elapsed since the last loss event**.

In classical protocols such as TCP Reno, the congestion window grows incrementally at each RTT, resulting in a step-like increase that is strongly influenced by the path round-trip time. CUBIC removes this direct dependency on RTT by making the window growth a function of absolute time, thus providing improved fairness among flows with different RTTs.

The behavior of CUBIC is described by a cubic function of time:

$$\text{cwnd}(t) = C \cdot (t - K)^3 + W_{\max}$$

where:

- $W_{\max}$ represents the congestion window size just before the last loss event;
- t is the time elapsed since the congestion event;
- K is the time instant at which the congestion window returns to $W_{\max}$;
- C is a constant controlling the aggressiveness of the window growth.

This formulation allows CUBIC to slow down the window growth when approaching $W_{\max}$, reducing oscillations, while increasing the sending rate more aggressively when moving away from the congestion point. Upon packet loss, CUBIC reduces the congestion window and enters a recovery phase, thus fully belonging to the class of loss-based congestion control algorithms.

B. TCP BBR

TCP BBR adopts a fundamentally different approach compared to loss-based algorithms such as CUBIC. In particular, **BBR does not use packet loss as the primary congestion signal**, but instead controls the sending rate based on an explicit estimation of the network path characteristics.

BBR maintains two key estimates:

- the **Bottleneck Bandwidth (BtlBw)**, representing the maximum achievable throughput along the path;
- the **minimum RTT (RTprop)**, which estimates the round-trip time in the absence of queueing.

Based on these estimates, BBR directly determines the amount of data to keep in flight, aiming to operate close to the **Bandwidth-Delay Product (BDP)** of the path:

$$\text{inflight} \approx \text{BtlBw} \times \text{RTprop}.$$

In this way, BBR proactively controls the sending rate through pacing, attempting to fully utilize the available bandwidth while avoiding the creation of persistent queues. Packet loss events may still occur, but they do not trigger the drastic rate reductions typical of loss-based algorithms.

As a result, BBR does not enter a traditional loss recovery phase and does not rely on CRB or SSRB-type policies, making it particularly robust in scenarios characterized by random, non-congestion-related packet loss.

III. Simulations on ns3

To perform the comparative experiments between TCP BBR and TCP CUBIC, representing the TCP stacks of Linux and Windows respectively, we employed the ns-3 network simulator, a discrete-event simulation framework widely adopted in academic research. Ns-3 supports the simulation of TCP congestion control algorithms, including representative models of TCP BBR and TCP CUBIC, enabling controlled and reproducible evaluation of their behavior under diverse network conditions. As we said before since TCP CUBIC can also be enabled on Linux, the results obtained in this NS3 simulations on Windows CUBIC are fully comparable to the one we could observe in Linux CUBIC.

A. Experiment A – Intra and Inter Protocol Fairness

In the first experiment, we evaluate the **intra-protocol fairness**, defined as the ability of a TCP congestion control algorithm to evenly share the available bandwidth among multiple flows using the **same algorithm** and competing over the same network bottleneck. We quantified fairness using the **Jain Fairness Index**, defined as:

$$J = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \cdot \sum_{i=1}^{n} x_i^2},$$

where x_i denotes the throughput of flow i and n is the number of considered flows. The index ranges from 0 to 1, with $J = 1$ indicating a perfectly fair bandwidth allocation.

1) Network Scenario: The experimental scenario follows a widely adopted model in the literature for fairness evaluation. We simulated in ns3 two client nodes simultaneously initiate identical downloads from a single server node, traversing an intermediate router that represents the network **bottleneck**. Since the capacity of the bottleneck link is insufficient to serve both flows at their maximum rate, the available bandwidth must be shared between them.

The goal of the experiment is to verify whether, under identical network conditions and congestion control algorithms, the two flows achieve similar bandwidth shares, indicating fair TCP behavior. For each algorithm, two **Client** $\rightarrow$ **Server** data flows are considered, starting at the same time and with identical characteristics. ACK flows in the reverse direction are not included in the fairness computation, as they do not represent competing data flows.

The network topology is illustrated below:

TABLE I: Intra-protocol fairness results

Alg.	C1 (Mb/s)	C2 (Mb/s)	Jain
BBR (Linux)	4.41	5.30	0.99
CUBIC (Win.)	5.12	4.82	1.00

TABLE II: Starbucks (bufferbloat) scenario results

Alg.	Thr. (Mb/s)	RTT (ms)
BBR (Linux)	4.83	81.9
CUBIC (Win.)	4.96	197.0

```
Client 1  ----\
               \
                ---- ROUTER ---- SERVER
               /
Client 2  ----/
```

The experiment is repeated separately for **BBR** and **CUBIC**, allowing the evaluation of intra-protocol fairness for each algorithm.

2) Experimental Results and Analysis: The results show that **both algorithms exhibit high intra-protocol fairness** in the considered scenario [Table I]. In particular, CUBIC achieves a Jain Fairness Index close to unity, indicating an almost perfectly symmetric bandwidth division between the two flows.

BBR also achieves a fairness index very close to 1, confirming that the algorithm can guarantee fair bandwidth sharing when competing exclusively with instances of the same protocol. The minor throughput differences observed between the two flows can be attributed to BBR's bandwidth estimation and probing mechanisms, which introduce transient variability compared to purely loss-based approaches, without compromising overall fairness.

The overall behavior is consistent with theoretical expectations for loss-based TCP protocols, which under symmetric conditions and long-lived flows tend to converge to an equitable bandwidth allocation.

3) Inter-protocol friendliness: We also conducted an additional analysis focusing on the behavior of two different TCP congestion control algorithms when they simultaneously compete for the same network resource. In this case, the goal is to observe how heterogeneous protocols **coexist and interact** under congestion, and whether one algorithm tends to systematically dominate the other. The network scenario is identical to that used for the intra-protocol fairness evaluation; however, the two flows employ **different congestion control algorithms**: one flow adopts **BBR** (Linux environment), while the other uses **CUBIC** (Windows environment). The two flows therefore compete directly for the same network resource, enabling the observation of their mutual behavior under contention

The measured average throughput of the BBR flow is 5.18 Mbps, while the CUBIC flow achieves an average throughput of 4.72 Mbps. This results in a BBR-to-CUBIC throughput ratio of approximately 1.10.

The results show a non-perfectly symmetric bandwidth allocation between the two flows, with BBR achieving a slightly higher throughput share than CUBIC. A BBR/CUBIC ratio close to 1.1 indicates that, in the considered scenario, BBR tends to occupy a larger fraction of the available capacity when directly competing with a loss-based algorithm such as CUBIC.

This behavior is consistent with the design differences between the two algorithms. CUBIC primarily adjusts its sending rate in response to loss-based congestion signals, reducing the congestion window upon packet loss. BBR, in contrast, follows a model-based approach, estimating the bottleneck bandwidth and the minimum RTT, and tends to maintain its sending rate close to the estimated value even in the presence of congestion. In direct competition scenarios, this can translate into a higher sustained throughput for BBR, particularly in the presence of buffers and shared congestion signals.

It is important to note that the observed imbalance remains limited and does not indicate excessively aggressive behavior by BBR.

B. Experiment b – Starbucks Scenario (Bufferbloat)

We tried to analyzes the behavior of the two TCP congestion control algorithms in the so-called *Starbucks scenario*, commonly used in the literature to model congested public networks characterized by **large buffers** and limited outbound capacity, simulating it in ns3. In such environments, the main issue is not bandwidth scarcity, but a significant increase in **latency** caused by packet accumulation in network queues, a phenomenon known as *bufferbloat*.

The goal of the experiment is to evaluate how different TCP algorithms react to deep queues, observing whether they tend to saturate available buffers, thereby increasing delay, or adopt a more conservative behavior aimed at limiting latency.

1) Network Scenario: The network scenario is intentionally simple in order to isolate the impact of bufferbloat. A single client initiates a download from a server through an intermediate router acting as the critical network element:

```
CLIENT ---- ROUTER ---- SERVER
```

The distinguishing feature lies in the router configuration. The incoming link operates at high capacity (100 Mbps), while the outgoing link toward the server is strongly constrained (5 Mbps). Additionally, the router is configured with a **very large queue**, capable of holding up to one thousand packets, and does not drop packets until the queue is completely full.

This configuration should induces rapid packet accumulation within the router buffer, causing a progressive increase in queueing delay and RTT, thus faithfully reproducing the conditions of congested public Wi-Fi networks.

TABLE III: Mobility scenario performance results

Alg.	Avg. Thr. (Mb/s)	Avg. RTT (ms)
BBR (Linux)	9.72	51.13
CUBIC (Win.)	9.91	48.44

2) Experimental Results and analysis: The measured average results are described in [Table II] As we can see, both algorithms achieve throughput values close to the bottleneck capacity, indicating effective bandwidth utilization. However, significant differences emerge in terms of latency.

CUBIC exhibits a substantially higher average RTT, exceeding 190 ms, indicating that the algorithm tends to fill the router queue before reducing its sending rate. This behavior is consistent with the loss-based nature of CUBIC, which primarily interprets congestion through packet loss rather than increasing delay. In the presence of large buffers, the absence of losses leads to persistent queue buildup and pronounced bufferbloat.

BBR, on the other hand, maintains a much lower average RTT while achieving comparable throughput. This result aligns with BBR's model-based approach, which estimates the bottleneck bandwidth and minimum RTT, limiting excess queue occupancy. Consequently, BBR is able to contain latency even in the presence of deep buffers.

C. Experiment c – Mobility Impact

We tried to analize the impact of **mobility** on the two TCP behavior, focusing on the ability of congestion control algorithms to adapt to simulated **dynamic variations of the round-trip time (RTT)**. Such variations are typical of real-world scenarios, including user movement, changes in distance from Wi-Fi access points, or handovers in cellular networks.

1) Network Scenario: The network scenario is designed to isolate the effect of RTT variations:

```
CLIENT (mobile) ---- ROUTER ---- SERVER
```

The client represents a mobile node, while router and server are fixed. The link delay is dynamically modified during the experiment to emulate mobility. Specifically, the RTT is initially set to 10 ms, then increased to 30 ms and finally to 60 ms, while keeping the link capacity constant.

Throughput and RTT metrics are computed as **average values over the entire experiment duration**, including both steady phases and transient periods.

2) Experimental Results and Analysis: Both algorithms maintain high average throughput despite RTT variations, with values close to link capacity. This indicates that both BBR and CUBIC are able, overall, to adapt to dynamic network conditions.

We can see that [Table III] overall, mobility-induced RTT variations introduce adaptation transients without significantly compromising TCP's ability to exploit available bandwidth, providing a baseline for more fine-grained per-phase analyses.

TABLE IV: Download performance results under ideal network conditions

Alg.	Completion Time (s)	Avg. Goodput (Mb/s)
BBR (Linux)	61.71	6.80
CUBIC (Win.)	47.23	8.88

D. Experiment d – Download Performance

We finally simulated the download performance TCP **download performance** under **ideal conditions**, with no external factors influencing congestion control behavior.

The objective is to provide a **baseline reference** by measuring download completion time and bandwidth utilization efficiency.

1) Network Scenario: The network topology is minimal:

```
Sender ---- Router ---- Receiver
```

Link capacity and propagation delay are constant throughout the experiment, with no packet loss or artificial congestion. Any performance difference can therefore be attributed solely to the congestion control strategy.

2) Experimental Results and Analisis: [Table IV]

CUBIC completes the download significantly faster and achieves higher goodput, exploiting available bandwidth more aggressively. This behavior is consistent with the loss-based nature of CUBIC, which rapidly increases the congestion window in the absence of congestion signals.

BBR exhibits longer completion time and lower goodput, reflecting its more conservative strategy aimed at operating close to the estimated bottleneck bandwidth. Under ideal conditions, this approach leads to lower performance compared to a more aggressive algorithm such as CUBIC.

This experiment provides a useful baseline for interpreting results in more complex scenarios, highlighting how TCP performance strongly depends on the network context in which the algorithm operates.

IV. Loss Detection and Loss Recovery

We already seen that in TCP it is essential to distinguish between **loss-based** and **bandwidth-based** algorithms, as this difference directly determines the mechanisms employed during the recovery phase following packet loss events.

Loss-based algorithms, such as **CUBIC**, interpret packet loss as the primary signal of congestion and trigger a **loss recovery** phase, during which the sending rate is reduced according to specific rate limitation policies.

In the context of the experiments presented in this work, the observed differences between Linux and Windows, when using the same congestion control algorithm, do not depend solely on the algorithm itself, but also on

the policies governing sender behavior during the recovery phase following packet loss.

In particular, more conservative recovery policies tend to drastically reduce the sender transmission rate, favoring pipe draining and increasing the likelihood of periods with near-zero throughput and fragmented recovery. Conversely, more permissive policies allow continuous transmission proportional to the received ACKs, enabling smoother recovery and reducing the probability of prolonged throughput collapses.

The analysis in this section is limited to scenarios in which **CUBIC** is used, as **BBR** does not rely on packet loss as a primary congestion signal and therefore does not follow traditional loss recovery mechanisms.

A. Loss Detection

Loss detection refers to the set of rules by which TCP determines whether a transmitted segment should be considered lost or simply delayed.

This mechanism differs from timeout-based recovery (RTO), where the absence of acknowledgments for a given segment leads to the assumption that all unacknowledged segments have been lost.

Loss detection decisions are critical, as they influence:

- the timing of retransmissions,
- the duration of the recovery phase,
- the likelihood of entering a timeout state.

An overly aggressive loss detection mechanism may trigger premature retransmissions, while an overly conservative one may unnecessarily delay flow recovery.

Modern TCP implementations on Linux rely on **RACK (Recent Acknowledgment)** as the primary loss detection mechanism.

RACK is based on temporal and ordering criteria: if a **more recent** segment has been acknowledged, while an older segment has not, the latter is considered likely lost.

In other words, RACK leverages information from recent acknowledgments to infer that an older segment is not merely delayed, but actually missing. This approach enables faster and more accurate loss detection, reducing reliance on heuristics based solely on duplicate ACK counting.

B. Proportional Rate Reduction (PRR)

PRR (Proportional Rate Reduction) is a TCP recovery mechanism that regulates **how much traffic the sender is allowed to transmit during the fast recovery phase**, i.e., after a loss has been detected by mechanisms such as RACK, but **before** a potential timeout occurs.

The core idea of PRR is that new data transmission during recovery should be **proportional to the progress of the flow**, as measured by received ACKs. This allows the sender to avoid overly aggressive transmission while still maintaining continuous data flow.

PRR does not determine when a loss is detected, nor how the congestion window evolves during steady-state operation. Its role is limited to **managing sender transmission during fast recovery**, providing a framework within which specific rate limitation policies can be applied.

Within this framework, the **Conservative Reduction Bound (CRB)** and **Slow start Rate Bound (SSRB)** policies define different approaches to limiting the sender transmission rate during recovery.

1) Conservative Reduction Bound (CRB): **CRB (Conservative Reduction Bound)** is a rate-limiting policy applied within PRR during fast recovery.

It defines a **conservative lower bound** on sender transmission, ensuring that a minimum amount of data continues to be sent in proportion to the received ACKs.

Under CRB, the rate reduction following a loss is **gradual and controlled**:

- the sender slows down,
- but avoids completely draining the pipe,
- maintaining continuous data flow even under adverse conditions.

From an observable standpoint, CRB typically results in:

- absence or rarity of zero-throughput periods,
- limited oscillations,
- increased flow stability during recovery.

CRB is **not a loss detection algorithm** and does not apply to timeout (RTO) events; it operates exclusively during fast recovery following loss detection.

2) Slow Start Reduction Bound (SSRB): The **Slow Start Reduction Bound (SSRB)** is an upper bound derived from the slow start behavior and represents a more restrictive limit than CRB on the amount of data that can be transmitted during fast recovery. In particular, SSRB limits how much of the sender's available sending capacity may be used when the recovery process follows slow-start-like dynamics, thereby constraining the amount of data kept in flight in a manner similar to pure slow start.

RFC 6937 formalizes SSRB as part of the sending control mechanism within **Proportional Rate Reduction (PRR)**. Under SSRB, the maximum amount of data that can be sent during fast recovery is bounded more tightly than under CRB, which is instead designed to be more permissive and to keep the pipe full in a smoother and more stable manner.

Both the SSRB and CRB bounds are computed based on the number of ACKs received and the size of the congestion window. Together, they define the admissible range of data that the sender is allowed to transmit before the recovery phase is completed.

From an empirical and statistical perspective, *SSRB-like* behaviors are typically associated with:

- higher throughput variability,
- increased standard deviation,

- a larger coefficient of variation,
- a higher likelihood of entering timeout conditions.

As with CRB, SSRB applies exclusively during the fast recovery phase and **does not govern TCP behavior in the case of retransmission timeouts (RTO)**.

In summary, PRR defines the general mechanism for regulating packet transmission during fast recovery, while CRB and SSRB represent two standardized reduction bounds within PRR, characterized respectively by a conservative and continuous sending behavior and by a more restrictive slow-start-based limitation of the sending rate.

V. Real Experiment for Recovery Policies

This section investigates the behavior of TCP congestion control recovery policies through a real-network experiment with controlled packet loss. The goal is to observe and compare how Windows and Linux TCP stacks react to non-congestive loss events, focusing on loss detection and loss recovery mechanisms at the sender side.

A. Experimental Setup

The experimental setup is designed to analyze the behavior of **TCP loss detection and loss recovery at the sender** under controlled packet loss conditions. The objective is to compare different TCP implementations and congestion control behaviors while keeping the network unchanged, thereby isolating the impact of the sender-side TCP stack.

The topology consists of an **external sender** (alternatively running Windows and Linux) connected to another host through a real network (LAN/Wi-Fi). On this host, a **Linux software router** is deployed, which forwards traffic through a virtual bottleneck link toward an **isolated receiver** implemented using a Linux network namespace on the same machine. The effective LAN throughput measured between the two hosts is approximately 24 Mbit/s.

1) Network Topology:

```
External Sender
(Windows / Linux)
        |
        |  Wi-Fi LAN
        v
Linux Software Router
(loss / delay injection)
        |
        |  Virtual link (bottleneck)
        v
Isolated Receiver
(Linux namespace)
```

Packet loss (0.5% and 2%) is introduced on the router–receiver link using a random, independent loss model, i.e., a fixed probability for each packet to be dropped. Propagation delay is also controlled on the same link. TCP traffic is generated using a continuous-flow application (`iperf3`) for a fixed duration.

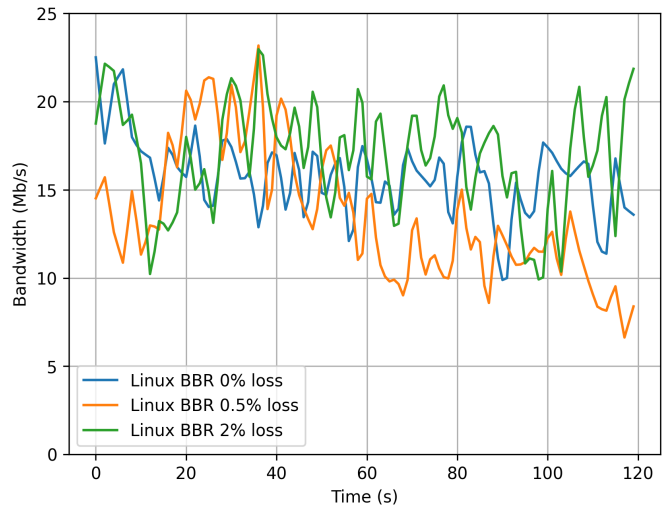

Fig. 1: Throughput over time comparison for TCP BBR under 2%,0.5% and 0% packet loss.

Packet traces are collected at the router, on the sender-facing interface, and analyzed offline to study the temporal evolution of the sending rate, highlighting rate stability, recovery phases, and potential timeout events.

B. Algorithm Selection

To focus specifically on recovery behavior, **BBR is excluded from this comparison**. As observed in the theory and confirmed through the experiment, BBR largely ignores random, non-congestive packet loss introduced on the outgoing link.

In fact [Fig.1], for all loss percentages, both the trend and the throughput values are very similar. This behavior is expected given BBR's design: packet loss is not treated as a primary congestion signal, the propagation delay remains constant (20 ms), and the minimum RTT estimate is stable. Since approximately 98–99.5% of packets are still delivered, the available bandwidth is not significantly reduced.

For this reason, the experiment is replicated using **TCP CUBIC on both Linux and Windows**, enabling a direct comparison of recovery policies. With Linux we used a **PRR + CRB** recovery strategy, while the default recovery policy used by Windows is not publicly documented and is therefore inferred from experimental observations.

C. Results and Analysis

To begin with, we tested the two protocols under 0% packet loss.

As can be observed in [Fig 2], also in this case the throughput values and the overall trends are extremely similar.

At a lower loss rate of 0.5%, TCP CUBIC exhibits significantly different behaviors for the two implementations under examination, as shown in [Fig 3].

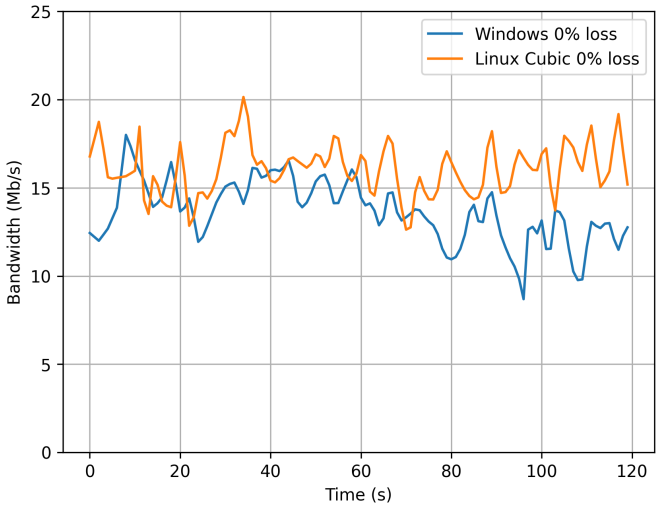

Fig. 2: Throughput over time comparison for Windows and Linux Cubic under 0% packet loss.

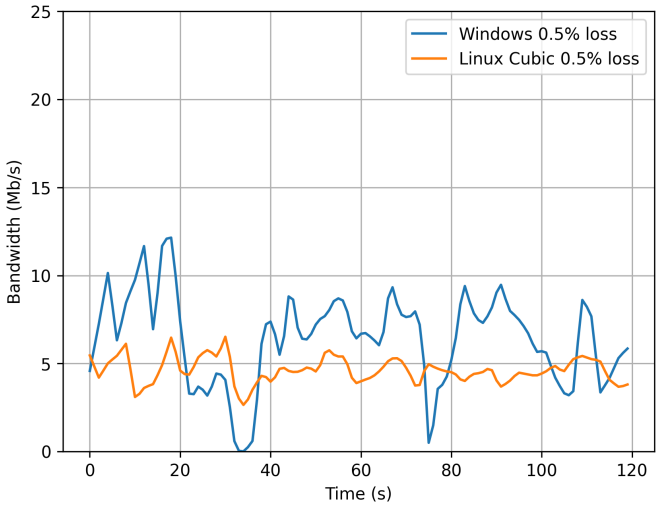

Fig. 3: Throughput over time comparison for Windows and Linux Cubic under 0.5% packet loss.

The Windows implementation achieves higher average throughput values, but at the cost of deeper and more frequent collapses, including periods of near-zero throughput. The Linux implementation, while achieving lower average throughput, shows greater stability and faster recovery following loss events.

For both implementations, we observe a reduction in average throughput compared to both the BBR results and the experiment conducted with 0% packet loss.

Under a 2% random packet loss rate, we observe the same behavior for TCP on Windows, but in a more pronounced form, along with an even lower average throughput for both implementations, as shown in [Fig 4]

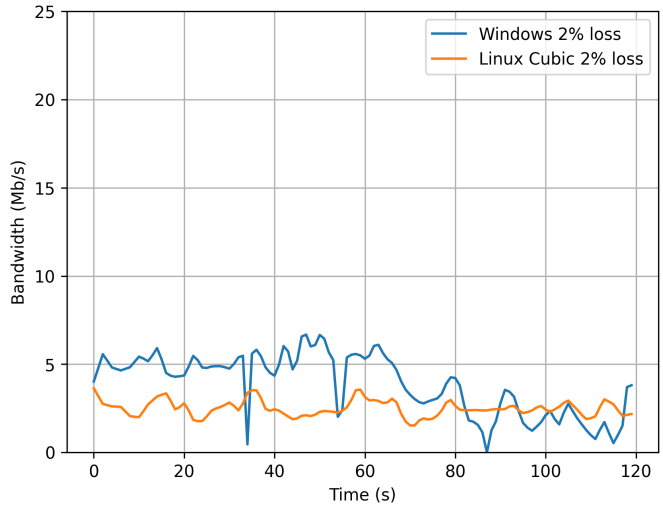

Fig. 4: Throughput over time comparison for Windows and Linux Cubic on 2% packet loss.

D. Recovery Policy Hypotheses

1) Hypothesis on Recovery Policies: Based on the available documentation, we assume that the Linux TCP implementation adopts a loss recovery policy based on Proportional Rate Reduction (PRR) with a Conservative Reduction Bound (CRB). Under this approach, following a packet loss event, the sender reduces its sending rate while continuing to transmit data proportionally to the received ACKs, thus avoiding pipe draining.

The experimental results appear to confirm this behavior: the observed throughput exhibits lower variability and is almost never zero under both 0.5% and 2% packet loss, indicating the presence of a continuous data flow even during recovery phases.

From a statistical perspective, this behavior is consistent with a CRB policy, which is characterized by smaller oscillations around the mean, lower standard deviation, and a reduced coefficient of variation. The observed data therefore support the hypothesis that the Linux implementation follows a CRB-based recovery strategy.

For Windows, it is not possible to directly identify the recovery policy adopted by the TCP stack. However, the behavior observed in the experimental data is compatible with a **strongly restrictive recovery behavior**, which is referred to in this work as *SSRB-like*.

In particular, the Windows results exhibit [Fig.3] [Fig.4]:

- instances of near-zero throughput (not observed in Linux),
- collapse events with zero throughput lasting for multiple samples,
- abrupt restart phases following recovery,
- pronounced throughput oscillations, more evident with a 2% packet loss rate.

From a statistical standpoint, these behaviors are more consistent with an SSRB-like dynamic, characterized by

large deviations from the mean, higher standard deviation, and an increased coefficient of variation. Both at 0.5% and 2% packet loss, the Windows implementation therefore exhibits a recovery behavior closer to a strongly restrictive recovery pattern than to a CRB-based policy.

References

[1] Microsoft Tech Community, "Windows Transport Converges on Two Congestion Providers: CUBIC and LEDBAT," 2018. [Online]. Available: https://techcommunity.microsoft.com/blog/networkingblog/windows-transport-converges-on-two-congestion-providers-cubic-and-ledbat/339819

[2] ns-3 Consortium, "ns-3 Network Simulator," [Online]. Available: https://www.nsnam.org/

[3] N. Dukkipati, M. Mathis, Y. Cheng, and A. Ghobadi, "Proportional Rate Reduction for TCP," RFC 6937, IETF, May 2013. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6937

[4] R. N. Shorten, D. J. Leith, and R. McKenna, "TCP Cubic," RFC 8312, IETF, February 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8312

[5] Ubuntu Manpage Repository, "tcpstat(1) – display TCP traffic statistics," Ubuntu Xenial Manpages. [Online]. Available: https://manpages.ubuntu.com/manpages/xenial/en/man1/tcpstat.1.html